can take an approach similar to the one you saw with the JBehave/Thucydides project in section 12.5.1, with a test-runner class to run the scenarios for a given tag or tags:

```
@Cucumber.Options(tags = {"@earning_points"},
                  format = {"json:target/cucumber/earning_points.json"})
public class EarningPoints {}
```

In this configuration, each parallel build job would be configured to run a different test runner, typically by passing a system property from the command line. Using Maven, for example, you could run something like this:

```
mvn test -Dtest=EarningPoints
```

Using JBehave, you'd typically use the `metafilter` system property directly:

```
mvn test -Dmetafilter=+earning_points
```

Virtually any modern CI tool will let you run build jobs in parallel like this, although the functionality (and even the vocabulary used) varies from tool to tool. In Bamboo, for example, you'd set up a build plan to deploy your application. This build plan would be made up of a number of stages: compilation and unit tests, integration tests, automated acceptance criteria, performance tests, and so forth. Within the automated acceptance criteria stage, you can run several jobs in parallel, with each job running a subset of the acceptance criteria.

If you're using Jenkins, you can use a multi-configuration project type for a more centralized approach. Multi-configuration projects let you run a build job multiple times with different parameters, either on the main build server or across a number of remote machines. The multi-configuration build job lets you define a list of parameter values to be passed to the automated build script, and it runs a separate build job for each parameter value. These build jobs are run in parallel, either on the master build server or on remote build machines.

Multi-configuration build jobs work well for automated acceptance criteria. For example, rather than having a separate build job for each tag you want to execute, you could use a multi-configuration build job and provide a set of values for the `test` or `metafilter` parameter. To do this, you'd first create a multi-configuration build job in Jenkins and add a User-Defined Axis in the Configuration Matrix section (see figure 12.10).

When Jenkins runs this multi-configuration build, it'll start a build job for each value you place in the User-Defined Axis field, passing in the values you define in the user-defined axis as system properties. So in the example in figure 12.10, it will start five child jobs respectively running the following commands:

```
mvn test -Dtest=Authentication
mvn test -Dtest=EarningPoints
mvn test -Dtest=StatusPoints
mvn test -Dtest=TransferringPoints
mvn test -Dtest=ViewingFlights
```